ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

РЕАЛИЗАЦИЯ ПОЛУАВТОМАТИЧЕСКИХ И АВТОМАТИЧЕСКИХ РЕЖИМОВ УПРАВЛЕНИЯ ОБОРУДОВАНИЕМ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков ограничения получения данных и приведения их к необходимому формату.

2 Задание и исходные данные

Наладить обмен данными между оборудованием внутри системы, создать условия переключения состояния оборудования. Взаимодействие элементов системы должно соответствовать предложенному в первой лабораторной работе проекту.

3 Ход выполнения задания

Добавлен класс Heater с полем мощности power и порогом включения switch_on_temperature. Метод auto_power(temperature) устанавливает power в «On», если температура ниже порога, иначе “Off”. После обработки команды температуры в маршруте GET /connect вызывается heater.auto_power с текущим значением env_sensor.temperature. В JSON-ответ добавлено поле heater_power. Для просмотра состояния обогревателя добавлен маршрут GET /connect_heater. Результаты работы представлены на рисунке 1.

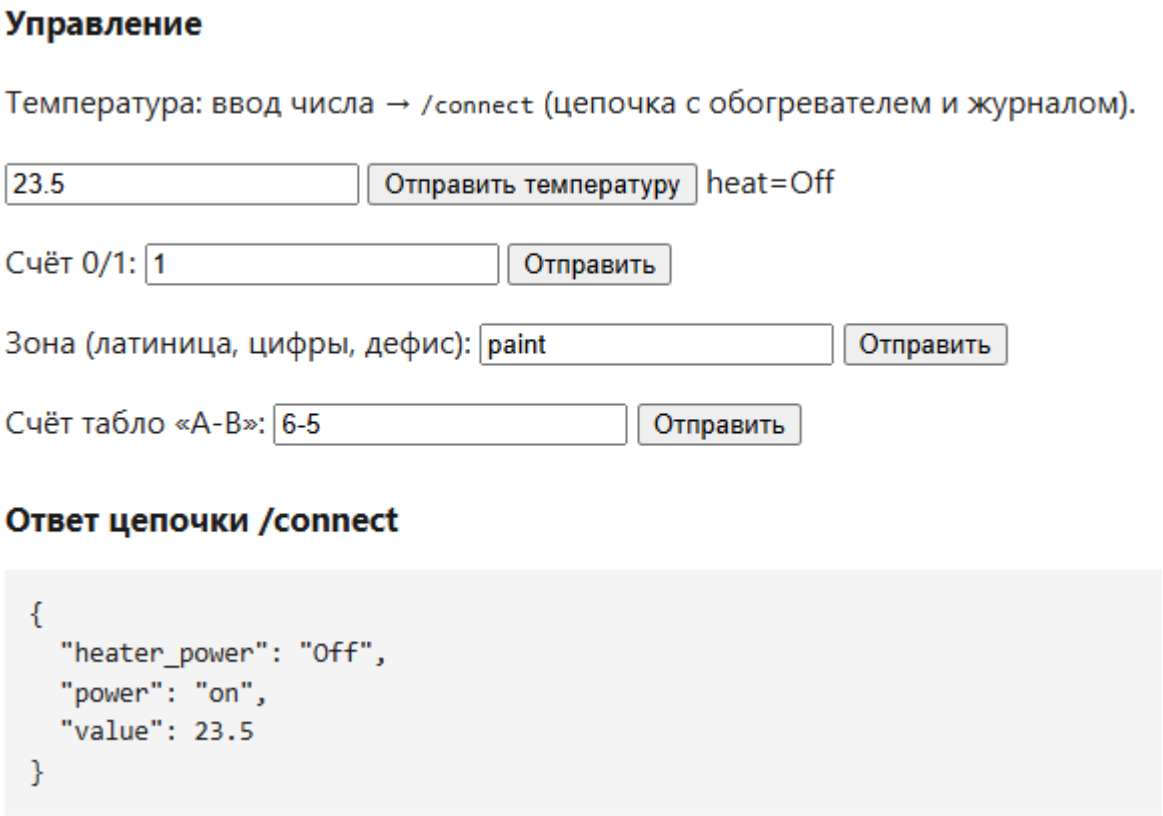

Рисунок 1 – Результат работы программы

4 Вывод

В ходе работы реализована связка “датчик микроклимата — обогреватель”: температура, задаваемая пользователем или приходящая из эмуляции, автоматически влияет на режим обогрева без отдельной команды на включение нагревателя.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
// ============================
// File: templates/sensor_emulator.html
// ============================
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <title>Эмуляторы вещей — лаб. 3–6</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}" />
    <script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
    <script src="{{ url_for('static', filename='js/index.js') }}"></script>
  </head>
  <body>
    <p><a href="{{ url_for('index') }}">← На главную</a></p>
    <h1>Мониторинг и команды (методичка, лаб. 3–6)</h1>

    <section>
      <h2>Мониторинг (лаб. 3: каждую секунду GET, отдельный ajax на вещь)</h2>
      <div class="row">
        <label>Температура °C <input type="text" id="mon_env" readonly /></label>
        <label>Счёт (гол) <input type="text" id="mon_score" readonly /></label>
      </div>
      <div class="row">
        <label>Зона <input type="text" id="mon_zone" readonly /></label>
        <label>Табло <input type="text" id="mon_board" readonly /></label>
      </div>
```

```
    </section>

    <section>
      <h2>Управление (лаб. 4–5: команды на сервер)</h2>
      <p>Температура: ввод числа → <code>/connect</code> (цепочка с обогревателем и журналом).</p>
      <input type="text" id="cmd_env" placeholder="23.5" />
      <button type="button" id="btn_env">Отправить температуру</button>
      <span id="cmd_env_reply"></span>

      <p>Счёт 0/1: <input type="text" id="cmd_score" placeholder="1" /> <button type="button" id="btn_score">Отправить</button></p>
      <p>Зона (латиница, цифры, дефис): <input type="text" id="cmd_zone" placeholder="paint" /> <button type="button" id="btn_zone">Отправить</button></p>
      <p>Счёт табло «A-B»: <input type="text" id="cmd_board" placeholder="12-10" /> <button type="button" id="btn_board">Отправить</button></p>
    </section>

    <section>
      <h2>Ответ цепочки /connect (лаб. 6–8)</h2>
      <pre id="chain_out"></pre>
    </section>
  </body>
</html>

// ============================
// File: app.py
// ============================
from __future__ import annotations
```

```
from flask import Flask, jsonify, render_template, request

import things

app = Flask(__name__)

env_sensor = things.EnvSensor("es1", "Микроклимат зала", 22.0, 400.0)
score_sensor = things.ScoreSensor("ss1", "Кольцо команды A", is_goal=False)
zone_sensor = things.ZoneSensor("zs1", "Разметка площадки", "three-point-arc")
scoreboard = things.Scoreboard("sb1", "Табло арены", 0, 0, 0)
heater = things.Heater("Обогреватель зала", 22.0)

store = things.CourtDataStore()
mcu = things.MainControlUnit(
    [score_sensor, zone_sensor, env_sensor, scoreboard],
    store,
    scoreboard,
)


def demo_iot_cycle() -> None:
    print("=== collectData ===")
    mcu.collectData()
    print("=== calculateScore ===")
    mcu.calculateScore()
    print("=== updateScoreboard ===")
    mcu.updateScoreboard()
    print("=== getRecords ===", len(store.getRecords()))
```

```
@app.route("/monitor/env")
def monitor_env():
    return jsonify(env_sensor.connect())


@app.route("/monitor/score")
def monitor_score():
    return jsonify(score_sensor.connect())


@app.route("/monitor/zone")
def monitor_zone():
    return jsonify(zone_sensor.connect())


@app.route("/monitor/board")
def monitor_board():
    return jsonify(scoreboard.connect())


@app.route("/connect")
def connect():
    response = env_sensor.connect_with_commands(request)
    heater.auto_power(env_sensor.temperature)
    response["heater_power"] = heater.power
    return jsonify(response)


@app.route("/connect_heater")
```

```
def connect_heater():
    return jsonify(heater.connect())


@app.route("/command/score")
def command_score():
    return jsonify(score_sensor.connect_with_commands(request))


@app.route("/command/zone")
def command_zone():
    return jsonify(zone_sensor.connect_with_commands(request))


@app.route("/command/board")
def command_board():
    return jsonify(scoreboard.connect_with_commands(request))


@app.route("/lab2")
def lab2_page():
    demo_iot_cycle()
    return render_template("lab2.html")


@app.route("/")
def index():
    return render_template("index.html")
```

```
@app.route("/emulator")
def emulator_page():
    return render_template("sensor_emulator.html")


if __name__ == "__main__":
    app.run(debug=True)

// ============================
// File: things.py
// ============================
from __future__ import annotations

import abc
import random
import re
from typing import Any, Dict, List, Optional

class Thing(abc.ABC):
    def __init__(self, thing_id: str, name: str) -> None:
        self.id = thing_id
        self.name = name

    @abc.abstractmethod
    def getStatus(self) -> str:
        pass

    def emulate(self) -> None:
        pass
```

```
    def connect(self, request: Any = None) -> Dict[str, Any]:
        """Лаб. 3–6: обмен с интерфейсом; возвращает dict для json.dumps в app.py."""
        raise NotImplementedError


class ScoreSensor(Thing):
    def __init__(self, thing_id: str, name: str, is_goal: bool = False) -> None:
        super().__init__(thing_id, name)
        self.is_goal = is_goal

    def getStatus(self) -> str:
        return "ScoreSensor[" + self.id + "]: is_goal=" + str(self.is_goal)

    def emulate(self) -> None:
        self.is_goal = random.choice([True, False])

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {"id": self.id, "value": 1 if self.is_goal else 0, "is_goal": self.is_goal}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        """Лаб. 4–5: команда из интерфейса (например коррекция)."""
        if request is not None:
            raw = request.args.get("value", "")
            try:
                v = int(float(raw))
                self.is_goal = bool(v)
            except ValueError:
```

```
            print("ScoreSensor: ожидалось число 0/1, получено:", type(raw).__name__)
        return {"power": "on", "is_goal": self.is_goal}


class ZoneSensor(Thing):
    ZONES = ("three-point-arc", "paint", "midcourt", "baseline")

    def __init__(self, thing_id: str, name: str, zone: str) -> None:
        super().__init__(thing_id, name)
        self.zone = zone

    def getStatus(self) -> str:
        return "ZoneSensor[" + self.id + "]: zone=" + self.zone

    def emulate(self) -> None:
        self.zone = random.choice(self.ZONES)

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {"id": self.id, "value": self.zone, "zone": self.zone}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        """Лаб. 5: строка — проверка регулярным выражением."""
        if request is None:
            return {"zone": self.zone}
        raw = request.args.get("value", "") or ""
        if re.fullmatch(r"[a-z0-9\-]+", raw):
            self.zone = raw
        else:
```

```
        print("ZoneSensor: недопустимая зона (нужны латиница, цифры, дефис):", repr(raw))
    return {"power": "on", "zone": self.zone}


class EnvSensor(Thing):
    def __init__(self, thing_id: str, name: str, temperature: float, light_lux: float) -> None:
        super().__init__(thing_id, name)
        self.temperature = float(temperature)
        self.lightLux = float(light_lux)

    def getStatus(self) -> str:
        return (
            "EnvSensor["
            + self.id
            + "]: t="
            + str(round(self.temperature, 1))
            + ", lux="
            + str(int(self.lightLux))
        )

    def emulate(self) -> None:
        self.temperature = round(random.uniform(18.0, 28.0), 1)
        self.lightLux = float(random.randint(300, 600))

    def connect(self, request: Any = None) -> Dict[str, Any]:
        """Мониторинг (лаб. 3): как Sensor в методичке — поле value для ajax."""
        self.emulate()
        return {
```

```
            "id": self.id,
            "value": self.temperature,
            "temperature": self.temperature,
            "lightLux": int(self.lightLux),
        }

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        """Лаб. 5–6: float из запроса (try/except как рис. 25)."""
        if request is None:
            return {"power": "on", "value": self.temperature}
        raw = request.args.get("value", "")
        try:
            self.temperature = float(raw)
            self.temperature = round(self.temperature, 1)
        except ValueError:
            print(
                "EnvSensor: не удалось привести к float, тип:",
                type(raw).__name__,
                "значение:",
                repr(raw),
            )
        return {"power": "on", "value": self.temperature}


class Scoreboard(Thing):
    def __init__(self, thing_id: str, name: str, score_a: int = 0, score_b: int = 0,
timer_sec: int = 0) -> None:
        super().__init__(thing_id, name)
        self.scoreA = score_a
        self.scoreB = score_b
```

```
    self.timerSec = timer_sec

def getStatus(self) -> str:
    return (
        "Scoreboard["
        + self.id
        + "]: "
        + str(self.scoreA)
        + ":"
        + str(self.scoreB)
        + ", t="
        + str(self.timerSec)
    )

def emulate(self) -> None:
    self.timerSec += random.randint(0, 2)
    if random.random() < 0.3:
        self.scoreA += 1

def connect(self, request: Any = None) -> Dict[str, Any]:
    self.emulate()
    return {
        "id": self.id,
        "value": self.scoreA,
        "scoreA": self.scoreA,
        "scoreB": self.scoreB,
        "timer": self.timerSec,
    }

def connect_with_commands(self, request: Any) -> Dict[str, Any]:
```

```
        if request is None:
            return {"scoreA": self.scoreA, "scoreB": self.scoreB}
        raw = request.args.get("value", "")
        try:
            parts = raw.split("-")
            if len(parts) == 2:
                self.scoreA = int(parts[0])
                self.scoreB = int(parts[1])
        except ValueError:
            print("Scoreboard: ожидался формат A-B для счёта")
        return {"power": "on", "scoreA": self.scoreA, "scoreB": self.scoreB}


class Heater:
    """Лаб. 6: обогреватель по температуре (класс Heater, рис. 27)."""

    def __init__(self, name: str, switch_on_temperature: float) -> None:
        self.name = name
        self.power = "Off"
        self.switch_on_temperature = switch_on_temperature

    def connect(self) -> Dict[str, Any]:
        return {"heater_power": self.power}

    def auto_power(self, temperature: float) -> None:
        self.power = "On" if temperature < self.switch_on_temperature else "Off"


class CourtDataStore:
    def __init__(self) -> None:
```

```
        self.events: List[str] = []
        self.telemetry: List[str] = []

    def saveEvent(self, data: Any) -> None:
        self.events.append("event=" + repr(data))

    def saveTelemetry(self, data: Any) -> None:
        self.telemetry.append("telemetry=" + repr(data))

    def getRecords(self) -> List[str]:
        return self.events + self.telemetry


class MainControlUnit:
    def __init__(self, things: List[Thing], store: CourtDataStore, scoreboard: Scoreboard) -> None:
        self.things = things
        self.store = store
        self.scoreboard = scoreboard

    def collectData(self) -> None:
        for t in self.things:
            self.store.saveTelemetry({"id": t.id, "status": t.getStatus()})

    def calculateScore(self) -> None:
        for t in self.things:
            if isinstance(t, ScoreSensor) and t.is_goal:
                self.scoreboard.scoreA += 1
                self.store.saveEvent({"type": "basket", "points": self.scoreboard.scoreA})
                break
```

```
    def updateScoreboard(self) -> None:
        self.scoreboard.timerSec += 1
// ============================
// File: static/js/index.js
// ============================
/* Лаб. 3: setInterval + ajax GET (методичка, рис. 16); лаб. 4 — отправка команд.
*/

function poll(url, id, pick) {
  $.ajax({
    type: "GET",
    url: url,
    dataType: "json",
    contentType: "application/json",
    data: {},
    success: function (response) {
      document.getElementById(id).value = pick(response);
    },
  });
}

function startMonitoring() {
  setInterval(function () {
    poll("/monitor/env", "mon_env", function (r) {
      return r.temperature != null ? r.temperature : r.value;
    });
  }, 1000);

  setInterval(function () {
```

```
    poll("/monitor/score", "mon_score", function (r) {
      return r.is_goal ? "гол" : "нет";
    });
  }, 1000);

  setInterval(function () {
    poll("/monitor/zone", "mon_zone", function (r) {
      return r.zone;
    });
  }, 1000);

  setInterval(function () {
    poll("/monitor/board", "mon_board", function (r) {
      return r.scoreA + ":" + r.scoreB + " t=" + r.timer;
    });
  }, 1000);
}

function sendCommand(url, valueFieldId, extra) {
  var v = document.getElementById(valueFieldId).value;
  $.ajax({
    type: "GET",
    url: url,
    dataType: "json",
    data: { value: v },
    success: function (response) {
      if (extra) extra(response);
    },
  });
}
```

```
$(function () {
  startMonitoring();

  $("#btn_env").on("click", function () {
    sendCommand("/connect", "cmd_env", function (r) {
      $("#cmd_env_reply").text("heat=" + r.heater_power);
      $("#chain_out").text(JSON.stringify(r, null, 2));
    });
  });

  $("#btn_score").on("click", function () {
    sendCommand("/command/score", "cmd_score");
  });
  $("#btn_zone").on("click", function () {
    sendCommand("/command/zone", "cmd_zone");
  });
  $("#btn_board").on("click", function () {
    sendCommand("/command/board", "cmd_board");
  });
});
```